

MC小组能力建设路标

——AI增强研发版

一、能力建设总体目标

随着AI Coding Agent能力持续增强，MC团队的研发模式将逐步从：

人工学习

→ 人工设计

→ 人工编码

→ 人工验证

→ 人工Debug

演进为：

人定义需求和架构

→ AI理解Spec与工程上下文

→ AI辅助设计/编码/验证

→ 工具形成确定性验证证据

→ AI独立Review

→ 人完成关键判断和最终Sign-off

因此，团队能力体系升级为：

****FPGA专业能力**

- MC业务技术能力
- AI增强研发能力
- 组织资产**

其中：

AI不是独立于研发之外的一项技能，而是横向嵌入FPGA和MC研发全过程。

二、AI时代FPGA能力的重新定义

传统FPGA工程师主要解决：

“我能不能把RTL写出来。”

AI时代则进一步要求：

能不能定义正确的问题、设计正确的架构、给AI建立正确的工程上下文，并判断AI结果是否可信。

因此传统基础能力不能削弱。

反而需要进一步加强：

- RTL
- Timing
- CDC / RDC
- Clock / Reset
- FPGA Architecture
- Resource
- Verification
- Debug

因为：

AI降低了“生成代码”的门槛，但提高了“判断代码是否正确”的要求。

三、AI增强FPGA开发能力

L1：AI辅助开发

AI可以承担：

- RTL模板
- FSM初版
- CSR
- Testbench
- Assertion初版
- Python / Tcl
- Code Explain
- Log分析
- 文档整理

工程师重点负责：

- Requirement
- Interface
- Clock / Reset
- Timing
- CDC
- Corner Case
- Review

目标：

AI成为每个FPGA工程师的基础研发助手。

四、AI参与模块完整开发

L2：模块级AI闭环

从：

AI帮我生成几段RTL

升级为：

AI参与完整模块开发生命周期。

标准流程：

Requirement

↓

AI理解需求

↓

AI生成Design Proposal

↓

Human Design Review

↓

AI生成RTL

↓

AI生成Test / Assertion

↓

Simulation / Regression

↓

AI分析Fail

↓

AI修正

↓

独立AI Review

↓

Human Sign-off

适合逐步应用于：

- Register
- Address Mapping
- Monitor
- Error Handling
- Debug Logic
- DFI Adapter部分模块
- Utility Module

五、Agent Ready FPGA工程能力

未来AI开发效率不仅取决于模型。

还取决于：

工程是不是容易被AI正确理解。

因此需要建设：

Agent Ready Repository

工程中明确：

- Architecture
- Directory Structure
- Module Boundary
- Module Ownership
- Build入口
- Simulation入口
- Regression入口
- Coding Rule
- Golden Baseline
- 禁止修改区域
- PASS / FAIL标准

目标：

新的AI Agent进入工程后，可以快速理解“项目是什么、该改哪里、不能改哪里、如何验证”。

这本身就是AI时代新的FPGA工程能力。

六、系统级AI辅助FPGA开发

L3：系统级AI

AI不再只分析单个文件。

而是分析完整链路：

Requirement
→ **Architecture**
→ **RTL**
→ **Verification**

例如：

Scheduler增加新的策略。

AI需要自动识别影响：

- Scheduler
- Bank Machine
- Timing Manager
- Command Encoder
- Data Path
- DFI
- Performance
- Verification

并输出：

- Impact Module
- Interface Impact
- Timing Impact
- Risk
- Regression Scope
- Test Case
- Assertion

目标：

从文件级AI升级为系统级AI。

七、AI增强MC业务能力

AI对MC的重要价值，不只是生成RTL。

更大的价值是：

将复杂、分散、容易遗漏的DDR协议知识结构化，并与设计和验证建立关联。

MC知识路径：

Spec

→ Requirement

→ Architecture

→ RTL

→ Verification

→ Evidence

八、Spec知识工程

传统方式：

人工阅读大量DDR / DFI规范。

AI时代进一步建设：

Spec

→ 结构化解析

→ Knowledge Base

→ Rule Database

→ Design / Verification Consumer

关键规则需要保存：

- Source
- Chapter
- Table / Figure
- Rule
- Condition
- Scope
- Exception
- RTL Consumer
- Verification Case

形成原则：

AI提出关键协议结论时，必须能够回溯到原始Spec证据。

九、Timing知识机器可读化

DDR Timing逐步从：

人脑中的协议知识

沉淀为：

机器可理解、可检查、可复用的规则。

例如：

ACT → RD

结构化记录：

- Timing: tRCD
- Scope
- Bank条件
- Rank条件
- Mode条件
- Exception
- RTL Implementation
- Assertion
- Test Case

未来可用于：

- RTL Review
- Assertion Generation
- Test Generation
- Coverage检查
- Spec Compliance检查

十、AI辅助MC架构分析

新增DDR功能时：

Spec Agent

负责识别：

- Command变化
- Timing变化
- MR变化
- State变化

↓

Architecture Agent

分析影响：

- Frontend
- Scheduler
- Bank Machine
- Timing Manager
- Encoder
- DFI
- Training
- Verification

最终形成：

Spec

→ **Requirement**

→ **Architecture**

→ **RTL**

→ **Verification**

完整Traceability。

十一、AI增强Verification

传统验证更多依赖：

工程师人工设计Test Case。

AI时代逐步转向：

Requirement Driven Verification。

每条Requirement建立：

Requirement

↓

RTL

↓

Assertion

↓

Test Case

↓

Regression Result

AI辅助：

- Requirement拆解
- Test Plan生成
- Corner Case搜索
- Assertion生成
- Coverage Gap分析
- Regression Fail聚类

但最终：

PASS / FAIL必须由确定性工具结果给出。

十二、AI增强Debug

AI特别适合：

搜索、关联、比较和缩小问题空间。

MC Fail出现以后：

AI读取：

- Log
- Command Trace
- State Trace
- Register
- Counter
- RTL
- Spec

完成：

Fail分类

例如：

- Protocol
- Timing
- Data Path

- DFI
- Training
- CDC
- FPGA Timing

↓

定位异常时间窗口

↓

关联

Command
→ Timing Rule
→ RTL State
→ Spec

↓

输出Root Cause Candidate

最后：

人负责验证因果关系并完成最终RCA。

十三、多Agent研发模式

L4：Agent化研发

复杂任务逐步由多个独立角色完成。

Spec Agent

负责协议和规范。

Architecture Agent

负责方案和影响分析。

RTL Agent

负责编码和重构。

Verification Agent

负责：

- Test
- Assertion
- Coverage

Review Agent

负责独立检查：

- Logic
- Reset
- CDC
- Timing Risk
- Corner Case

RCA Agent

负责：

- Fail聚类
- Evidence关联
- Root Cause候选

形成：

AI分析

↓

AI实现

↓

工具验证

↓

独立AI Review

↓

Human Sign-off

十四、AI时代需要进一步加强的人类能力

AI越强，以下能力越重要。

1. 第一性原理

工程师必须知道：

DRAM为什么这样工作。

否则无法识别AI幻觉。

2. Architecture

人负责：

- 系统边界
- 模块划分
- Trade-off

3. Protocol Interpretation

关键协议解释不能完全交给AI。

4. Timing / CDC / Reset

以下属于高风险领域：

- Clock Architecture
- Timing Exception
- CDC / RDC
- Reset Sequence

必须强化人工Review。

5. Risk Judgment

AI可以发现风险。

最终：

风险是否接受由人决定。

十五、组织资产建设

AI时代的组织资产定义为：

团队在项目实践中沉淀形成、可持续复用和传承的技术成果、知识体系、AI能力、研发方法及人才能力。

重点包括以下六类。

1. 设计资产

沉淀：

- MC Architecture
- Design Spec
- Interface Spec
- RTL
- FPGA公共IP
- Clock / Reset框架
- Debug框架

2. 验证资产

沉淀：

- Testbench
- Test Case
- Assertion
- Coverage
- Regression
- Golden Case
- Error Injection
- Performance Benchmark

原则：

AI可以修改实现，但不能随意修改判卷标准。

3. 知识资产

沉淀：

- DDR Spec Knowledge Base
- DFI Knowledge Base
- Timing Rule Database
- Architecture Knowledge
- Issue / RCA
- Debug经验
- Performance Case

要求：

关键知识必须能够回溯原始证据。

4. AI研发资产

沉淀：

- Agent Ready Repo
- AI Task Template
- Prompt
- Skill
- Spec Agent
- RTL Agent
- Verification Agent
- Review Agent
- RCA Agent
- AI Benchmark

AI能力从：

个人使用技巧

升级为：

团队可重复调用的组织能力。

5. 研发方法资产

建立标准流程：

- Architecture Review
- Design Review
- RTL Review
- Timing / CDC Review
- AI Review
- Verification Review
- RCA
- Technical Decision Record
- Project Review

建立AI任务风险分级：

Low Risk

AI可较自主完成：

- 文档
- Script
- Log分类
- Test模板

Medium Risk

AI实现 + 人工Review:

- 普通RTL
- Testbench
- Assertion

High Risk

AI只能辅助:

- Architecture
 - Scheduler策略
 - Timing Rule
 - CDC
 - Timing Exception
 - Protocol Interpretation
-

6. 人才能力资产

传统人才梯队进一步升级为:

Level 1: AI增强模块开发者

能够:

- 使用AI
- Review AI代码
- 执行验证
- 定位基础问题

Level 2: 模块Owner

能够:

- 定义Requirement
- 定义Design
- 驱动AI完成开发
- 建立Verification
- 完成Sign-off

Level 3: Subsystem Owner

能够管理:

- 多个模块
- 多个Agent
- 跨模块问题

Level 4: MC Architect

重点负责：

- Architecture
- Protocol
- Performance
- Risk
- Technical Decision

同时形成：

- Owner
 - Backup
 - Skill Matrix
 - Mentor
 - 新人培养路径
 - AI Methodology Owner
-

十六、AI研发质量门禁

禁止：

AI自己生成 → AI自己判断PASS → 直接提交。

标准流程：

Agent A实现

↓

Golden Tool验证

↓

Agent B独立Review

↓

Human Review

↓

Sign-off

对于：

- Spec

- Timing
- CDC
- Training
- Scheduler
- Architecture

必须使用更严格的证据和签核机制。

十七、阶段路标

0~3个月：AI辅助开发标准化

完成：

- AI使用规范
- AI Task Template
- Spec知识库
- RTL Review模板
- Regression分析模板

同时补齐：

- FPGA Timing
 - CDC / RDC
 - DDR Command
 - DDR Timing
 - MC Architecture
 - DFI
-

3~6个月：AI参与模块闭环

选择低风险模块建立：

Requirement

→ Design

→ RTL

→ Verification

→ Review

完整流程。

同步形成：

- Agent Ready Repo
 - Golden Regression
 - AI Review Gate
-

6~12个月：系统级AI研发

逐步建立：

- Spec Agent
- RTL Agent
- Verification Agent
- Review Agent

应用到：

- Bank Machine
- Timing Manager
- Refresh
- DFI
- Data Path

形成：

AI参与的端到端研发流程。

12个月以后：AI-Native MC研发平台

最终形成：

Spec

↓

Requirement

↓

Architecture

↓

RTL Agent

↓

Verification Agent

↓

Regression

↓

Review Agent

↓

Human Sign-off

↓

Knowledge Feedback

形成闭环。

十八、最终建设目标

传统能力建设重点解决：

团队有没有能力把MC做出来。

AI时代能力建设进一步解决：

如何将专家经验、规范知识、设计方法、验证方法和AI Agent固化为组织能力，让研发效率和质量随着项目不断积累，而不是每个项目重新开始。

最终形成：

人负责原理、架构和判断

+

AI负责规模化分析和执行

+

工具负责确定性验证

+

组织资产保证能力持续沉淀和复制

推动MC团队从：

传统人力密集型研发

逐步升级为：

AI增强型、知识密集型研发团队。